

Hospital Priority Queue: Max Heap vs Heap Sort vs Quick Sort

Question 1 — A hospital manages patients by severity (higher score = higher priority).

Input data (input.txt): 45, 72, 30, 90, 65, 50, 85

Files

File	Description
hospital_priority.c	C source code (Max Heap, Heap Sort, Quick Sort)
input.txt	Input severity scores
output.txt	Program output
README.md	Trace tables, complexity analysis, comparison, conclusion

Compile and run: `gcc hospital_priority.c -o hp && ./hp`

(a) Max Heap: Trace Table (insertion)

Step	Insert	Heap after insertion
1	45	[45]
2	72	[72, 45]
3	30	[72, 45, 30]
4	90	[90, 72, 30, 45]
5	65	[90, 72, 30, 45, 65]
6	50	[90, 72, 50, 45, 65, 30]
7	85	[90, 72, 85, 45, 65, 30, 50]

Heap height = 2, comparisons = 9, swaps = 5.

(b) Heap Sort: Trace Table

Input: [45, 72, 30, 90, 65, 50, 85]

Step	Action	Array
0	Build max heap	[90, 72, 85, 45, 65, 50, 30]
1	Extract 90	[85, 72, 50, 45, 65, 30, 90]
2	Extract 85	[72, 65, 50, 45, 30, 85, 90]
3	Extract 72	[65, 45, 50, 30, 72, 85, 90]
4	Extract 65	[50, 45, 30, 65, 72, 85, 90]
5	Extract 50	[45, 30, 50, 65, 72, 85, 90]
6	Extract 45	[30, 45, 50, 65, 72, 85, 90]

Sorted: [30, 45, 50, 65, 72, 85, 90]. Comparisons = 21, swaps = 18.

(b) Quick Sort: Trace Table (last element as pivot)

Step	Pivot	Range	Array after partition
1	85	0..6	[45, 72, 30, 65, 50, 85, 90]
2	50	0..4	[45, 30, 50, 65, 72, 85, 90]

Step	Pivot	Range	Array after partition
3	30	0..1	[30, 45, 50, 65, 72, 85, 90]
4	72	3..4	[30, 45, 50, 65, 72, 85, 90]

Sorted: [30, 45, 50, 65, 72, 85, 90]. Comparisons = 12, swaps = 6.

(c) Complexity Analysis

Operation	Time	Space
Max Heap insert	$O(\log n)$	$O(n)$ to store heap
Max Heap get highest priority (peek)	$O(1)$	$O(1)$
Max Heap remove highest priority	$O(\log n)$	$O(1)$
Build heap	$O(n)$	$O(1)$
Heap Sort	$O(n \log n)$ best, average and worst	$O(1)$ extra (in-place)
Quick Sort	$O(n \log n)$ best/average, $O(n^2)$ worst	$O(\log n)$ stack average, $O(n)$ worst

Comparison Table

Criterion	Max Heap / Heap Sort	Quick Sort
Structure	Complete binary tree, height $\text{floor}(\log_2 n) = 2$	No persistent structure; recursion depth about 3-4
Comparisons ($n=7$)	21 (sort), 9 (insertion only)	12
Swaps ($n=7$)	18 (sort), 5 (insertion only)	6
New patient arrives	$O(\log n)$	Re-sort needed: $O(n \log n)$
Highest-priority patient	$O(1)$	$O(1)$ only after sorting
Worst case	$O(n \log n)$	$O(n^2)$
Extra space	$O(1)$	$O(\log n)$ to $O(n)$

Final Conclusion

For a one-time sort of a fixed list, Quick Sort used fewer comparisons and swaps (12 and 6 vs 21 and 18) on this input. However, the hospital **continuously inserts patients and needs the highest-priority patient immediately**. A Max Heap supports this best: each new patient is inserted in $O(\log n)$, the most critical patient is available in $O(1)$, and removal takes $O(\log n)$. Quick Sort would need a full re-sort of $O(n \log n)$ after every arrival, and its $O(n^2)$ worst case is risky in a critical setting. **The Max Heap (priority queue) is the more suitable approach.**